



03. Nested Documents & Deletion Operations

Nested Documents & Dot Notation

MongoDB documents ke andar child objects ya arrays store kiye ja sakte hain. Nested fields ko query karne ke liye **Dot Notation** use hota hai (hamesha quotes ke andar!):

```
mongosh - Nested Query

// Document structure: { name: "Aman", performance: { marks: 88, grade: "A" } }
db.student.find({ "performance.marks": { $gte: 85 } });
// Quotes ( " ") lagana mandatory hai jab field me dot (.) ho!
```



Deletion Operations

Command	Purpose	Return Value
db.student.deleteOne(filter)	Pehla matched document delete karta hai.	{ acknowledged: true, deletedCount: 1 }
db.student.deleteMany(filter)	Saare matched documents delete karta hai.	{ acknowledged: true, deletedCount: N }
db.student.drop()	Puri 'student' collection delete kar deta hai.	true
db.dropDatabase()	Currently active database ko permanently drop karta hai.	{ ok: 1, dropped: "college" }

```
mongosh - Deletions

// 1. Delete single document
db.student.deleteOne({ name: "Amit Patel" });

// 2. Delete all students with marks Less than 40
db.student.deleteMany({ marks: { $lt: 40 } });
```



CAUTION: deleteMany({}) khali filter ke saath run karne par collection ke saare documents delete ho jayenge!



Introduction to Mongoose ODM & Connection



What is Mongoose ODM?

Mongoose ek Object Data Modeling (ODM) library hai Node.js ke liye. Ye JavaScript objects aur MongoDB documents ke beech ek translation aur validation layer banata hai.

Node.js App
(JS Objects)

⇌ Mongoose ODM ⇌
(Schema, Validations, Casting)

MongoDB Server
(BSON Documents)



Connection String Anatomy

mongodb://127.0.0.1:27017/collegeDB

Protocol | Host IP (localhost) | Default Port | Database Name



Connecting: Promises vs Async/Await

```
index.js - Mongoose Connection

const mongoose = require("mongoose");

// Recommended Industry Standard: Async/Await
async function main() {
  await mongoose.connect("mongodb://127.0.0.1:27017/collegeDB");
}

main()
  .then(() => console.log("✅ MongoDB Connected Successfully!"))
  .catch((err) => console.log("❌ Connection Error:", err));
```



WHY NOT NATIVE DRIVER? Native MongoDB driver me koi strict schema nahi hota, jisse typos aur invalid data save hone ka khatra rehta hai. Mongoose strict schema enforcement deta hai!